

Chapter 37: Angular Elements

1. Overview

Angular Elements is the package (`@angular/elements`) that lets you package an Angular component as a **native Custom Element** (Web Component, per the Custom Elements v1 spec). Once packaged, that component becomes a plain DOM tag — say `<my-widget>` — that can be dropped into *any* HTML page, regardless of whether the host page is built with Angular, React, Vue, jQuery, a static site generator, or a CMS template (WordPress, Drupal, Adobe Experience Manager, Salesforce, SharePoint, etc.).

The core API is a single function:

```
createCustomElement(component: Type<any>, config: NgElementConfig):  
NgElementConstructor<any>
```

It takes an Angular component class plus an injector, and returns a constructor you register with the browser's native `customElements.define()` registry. From that point on, the browser itself is responsible for instantiating your Angular component whenever it encounters the tag in the DOM — no Angular router, no Angular bootstrap of a full app shell required by the *host* page.

Why this exists — the core use cases

- **Embedding Angular widgets in non-Angular apps.** A React or Vue dashboard needs one complex chart/grid component that already exists as Angular — instead of a rewrite, ship it as a custom element.
- **Legacy application modernization.** A jQuery/ASP.NET MVC/Rails monolith can incrementally adopt Angular one widget at a time, without a full framework migration ("strangler fig" pattern).
- **CMS and marketing page embedding.** Content editors in a CMS (no build pipeline, no npm) can paste a `<script>` tag and a custom HTML tag into a page body — e.g., a lead-gen form, a pricing calculator, a product configurator.
- **Design-system distribution.** Publishing a set of framework-agnostic UI primitives that any team, on any stack, can consume as plain HTML tags.
- **Micro-frontends.** Different teams ship independently versioned/deployed Angular apps as isolated custom elements composed on one shell page.

The key conceptual shift: normally an Angular **component** only exists inside the Angular component tree, instantiated by the Angular compiler/renderer as part of a module/standalone bootstrap. An Angular **element** exists inside the **browser's native element registry** — it is instantiated by the browser's HTML parser/DOM APIs the moment the tag appears, exactly like `<video>` or `<select>`.

2. Core Concepts

2.1 The Custom Elements v1 spec (browser-native contract)

A custom element is any class that extends `HTMLElement` and is registered via:

```
customElements.define('tag-name', MyElementClass);
```

The spec requires/allows these reserved lifecycle callback methods on the class:

Callback	When browser calls it
<code>constructor()</code>	Element created (via <code>new</code> or parser upgrade). Must not touch attributes/children yet.
<code>connectedCallback()</code>	Element inserted into the DOM (a <i>live</i> document). Can be called multiple times if moved.
<code>disconnectedCallback()</code>	Element removed from the DOM.
<code>attributeChangedCallback(name, oldVal, newVal)</code>	An attribute listed in static <code>observedAttributes</code> changed.
<code>adoptedCallback()</code>	Element moved to a new <code>document</code> (rare — e.g., <code>document.adoptNode()</code>).

Angular Elements' `createCustomElement()` generates a class implementing exactly this contract, and internally maps each callback to the corresponding piece of Angular's component lifecycle and change-detection machinery.

2.2 `createCustomElement` and `NgElementConstructor`

```
function createCustomElement<P>(  
  component: Type<P>,  
  config: NgElementConfig  
) : NgElementConstructor<P>
```

- `component`: any Angular component class (standalone or declared in an `NgModule`).
- `config: { injector: Injector }`: supplies the injector the component's own component-injector will be created from — this is how the element gets access to Angular DI (services, `HttpClient`, tokens, etc.) even though it's living outside any Angular application context that the host page controls.
- Returns `NgElementConstructor<P>`, a class type extending `HTMLElement` with an additional generic property:
 - `.observedAttributes: string[]` (static) — derived automatically from the component's `@Input()` metadata.

You never call `new NgElementConstructor()` directly in normal usage — you pass it to `customElements.define(tagName, NgElementConstructor)`, and the browser instantiates it.

2.3 Attribute-to-`@Input()` mapping

Angular components use camelCase `@Input()` property names (`maxValue`, `userId`). HTML attributes are conventionally lowercase/kebab-case and are always strings. Angular Elements bridges this automatically:

- For each `@Input()` on the component, Angular Elements computes a corresponding **attribute name** using kebab-case conversion (`maxValue` → `max-value`).
- These names are exposed as the class's static `observedAttributes` array, which is what the browser's

parser watches for `attributeChangedCallback`.

- When an attribute changes (either set in markup, or via `e1.setAttribute(...)`), Angular Elements:
 1. Converts the attribute name back to the input's property name.
 2. Passes the **new attribute value as a string** (no automatic type coercion to number/boolean/object — this is a common interview trap, covered in §5).
 3. Sets it onto the underlying component instance's input property.
 4. Marks the component for change detection (see §4).
- You can also set the **property directly** in JS — `document.querySelector('my-widget').userId = 42` — which bypasses the string-only attribute path and preserves the real type. This is the recommended way to pass non-string data (objects, arrays, numbers, booleans) into an Angular Element.

2.4 Outputs → `CustomEvent`

`@Output()` EventEmitters on the wrapped component are automatically translated into native DOM `CustomEvent`s dispatched on the host element, using the (kebab-cased, by convention, though Angular actually keeps the original name) output property name as the event type. The host page listens with standard `addEventListener`, not Angular's `(event)` binding syntax — because the host page may not even have Angular. The emitted value is available on `event.detail`.

2.5 Standalone bundling model

Since Angular v14+ standalone components, and especially from v15+, Angular Elements is most commonly built from a **standalone component** with its own dedicated entry point, bundled independently of the rest of the app. Key architectural points:

- Each custom element bundle typically ships its **own copy of the Angular runtime** (core, common, the renderer, zone.js unless zoneless) unless you deliberately architect shared externals — there is no "one Angular runtime per page" model by default like there would be within a single Angular application.
- You can use the Angular CLI/ `ng build` with `output: 'static'` /custom Webpack or esbuild config to produce a **single self-contained JS file** (inlining templates/styles) suitable for a `<script>` tag drop-in, historically done via `ngx-build-plus` or custom esbuild bundling, and natively supported more directly via Angular's application builder producing a single chunk.
- `zone.js`, if used, must be loaded exactly once globally on the host page — loading it twice (e.g., two independently-bundled elements each bundling zone.js) throws "Zone already loaded" errors. Standalone/zoneless (`provideExperimentalZonelessChangeDetection` / signals-based) elements sidestep this.

2.6 Shadow DOM and view encapsulation

By default, Angular components use `viewEncapsulation.Emulated` — CSS scoping is faked via attribute selectors (`_ngcontent-xxx`), not real Shadow DOM. When packaging as a custom element you frequently want **real** encapsulation so the element's styles can't leak into/from the host page CSS. This means explicitly setting:

```
@Component({
  selector: 'app-widget',
  encapsulation: ViewEncapsulation.ShadowDom,
  ...
})
```

With `ViewEncapsulation.ShadowDom`, Angular attaches a genuine `shadowRoot` to the host element and renders the component's template/styles inside it — giving true CSS and (mostly) DOM isolation, matching what you'd expect from a "real" Web Component. Without this, an Angular Element still works, but its styles are just global `<style>` tags in the document `<head>` like any normal Angular app, and can bleed into/be bled into by the host page's own CSS.

3. Code Examples

3.1 The wrapped component (standalone)

```
// popup.component.ts
import { Component, Input, Output, EventEmitter, ViewEncapsulation } from '@angular/core';

@Component({
  selector: 'app-popup',
  standalone: true,
  encapsulation: ViewEncapsulation.ShadowDom,
  template: `
    <div class="popup" *ngIf="visible">
      <h3>{{ title }}</h3>
      <p>{{ message }}</p>
      <button (click)="close.emit('closed-by-user')">Close</button>
    </div>
  `,
  styles: [
    `.popup { border: 1px solid #333; padding: 1rem; background: #fff; }`
  ],
})
export class PopupComponent {
  @Input() title = 'Notice';
  @Input() message = '';
  @Input() visible = true;

  @Output() close = new EventEmitter<string>();
}
```

Note: with a standalone component you must import any structural directives it needs (`*ngIf` requires `CommonModule` or `NgIf` in the component's `imports` array) since there's no enclosing `NgModule` supplying them implicitly.

```
// popup.component.ts (imports fix)
import { Component, Input, Output, EventEmitter, ViewEncapsulation } from '@angular/core';
import { NgIf } from '@angular/common';

@Component({
  selector: 'app-popup',
  standalone: true,
  imports: [NgIf],
  encapsulation: ViewEncapsulation.ShadowDom,
  template: `...`,
  styles: [`...`],
})
export class PopupComponent { /* ... */ }
```

3.2 Registering the custom element in main.ts

```
// main.ts
import { createApplication } from '@angular/platform-browser';
import { createCustomElement } from '@angular/elements';
import { PopupComponent } from './app/popup.component';

(async () => {
  // createApplication bootstraps an Angular "application ref" + root injector
  // WITHOUT mounting any component into the DOM – perfect for Elements.
  const app = await createApplication({
    providers: [
      // any app-wide providers the element needs: HttpClient, custom services, etc.
    ],
  });

  const injector = app.injector;

  const PopupElement = createCustomElement(PopupComponent, { injector });

  customElements.define('app-popup', PopupElement);
})();
```

Older (pre-standalone-APIs / NgModule-based) pattern, still valid and commonly asked about in interviews:

```
// app.module.ts
import { NgModule, Injector, DoBootstrap } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';
import { createCustomElement } from '@angular/elements';
import { PopupComponent } from './popup.component';

@NgModule({
  declarations: [PopupComponent],
  imports: [BrowserModule],
  // entryComponents was required pre-Ivy; not needed since Ivy (Angular 9+)
})
export class AppModule implements DoBootstrap {
```

```

constructor(private injector: Injector) {}

ngDoBootstrap() {
  const PopupElement = createCustomElement(PopupComponent, { injector: this.injector });
  customElements.define('app-popup', PopupElement);
  // Deliberately NOT calling this.bootstrapModule(AppComponent) here –
  // DoBootstrap lets us skip the normal root-component bootstrap entirely,
  // since our "root" is the custom element registry, not an AppComponent.
}
}

```

```

// main.ts (NgModule variant)
import { platformBrowserDynamic } from '@angular/platform-browser-dynamic';
import { AppModule } from './app/app.module';

platformBrowserDynamic().bootstrapModule(AppModule);

```

3.3 Consuming it from plain, non-Angular HTML

```

<!DOCTYPE html>
<html>
<head>
  <title>Legacy CMS Page</title>
  <!-- The bundled Angular Element script – self-contained, includes Angular runtime -->
  <script src="popup-element.bundle.js" defer></script>
</head>
<body>
  <h1>welcome to our legacy jQuery site</h1>

  <!-- Attributes map to @Input()s automatically, kebab-cased -->
  <app-popup title="Cookie Notice" message="We use cookies." visible="true"></app-popup>

  <script>
    document.addEventListener('DOMContentLoaded', () => {
      const popup = document.querySelector('app-popup');

      // Listening to an Angular @Output() as a native CustomEvent
      popup.addEventListener('close', (event) => {
        console.log('Angular element said:', event.detail); // "closed-by-user"
        popup.remove(); // plain DOM manipulation, no Angular needed
      });

      // Setting a non-string input via the JS property (not the string attribute)
      // preserves real types, e.g. an object or boolean.
      popup.visible = false; // real boolean, not the string "false"
    });
  </script>
</body>
</html>

```

Important gotcha demonstrated above: `visible="true"` as an HTML attribute is passed to Angular as the **string** `"true"`, not the boolean `true`. If your component logic does `if (this.visible)`, the string `"false"` is truthy in JS and would evaluate as `true`! Always assign booleans/numbers/objects via the **DOM property**, not the HTML attribute, when precision matters (see §5).

3.4 Consuming from a React app

```
import React, { useRef, useEffect } from 'react';
import './popup-element.bundle.js'; // side-effect import registers the custom element

function App() {
  const ref = useRef(null);

  useEffect(() => {
    const el = ref.current;
    const handler = (e) => console.log('closed with', e.detail);
    el.addEventListener('close', handler);
    return () => el.removeEventListener('close', handler);
  }, []);

  return <app-popup ref={ref} title="From React" message="Hi!" />;
}
```

4. Internal Working

4.1 What `createCustomElement` actually builds

`createCustomElement(component, config)` returns a dynamically generated class, roughly (conceptually) shaped like:

```
class NgElementImpl extends NgElement {
  static readonly observedAttributes = /* kebab-cased @Input() attribute names */;

  constructor() {
    super();
    // NOT creating the Angular component yet – constructor must stay lightweight per spec.
  }

  connectedCallback() {
    // 1. Lazily create a new NgElementStrategy the first time the element connects.
    // 2. The strategy creates a *new, isolated* component instance via
    //    the injector passed to createCustomElement, using
    //    ComponentFactory (pre-Ivy) / createComponent (Ivy) APIs.
    // 3. Attaches the resulting component's host DOM node as this custom element's
    //    content (or into its shadowRoot, if ViewEncapsulation.ShadowDom).
    // 4. Applies any attributes/properties that were already set before connection.
    // 5. Triggers Angular's initial change detection for this component subtree.
  }

  disconnectedCallback() {
```

```

    // Destroys the component instance (calls ngOnDestroy), tears down subscriptions,
    // and detaches it from Angular's ApplicationRef so it stops being change-detected.
  }

  attributeChangedCallback(attrName, oldValue, newValue) {
    // Converts kebab-case attrName back to the camelCase @Input() property name,
    // sets `component.instance[inputPropName] = newValue` (as a string),
    // and schedules/triggers change detection (markDirty / detectChanges) so the
    // update is reflected in the rendered view synchronously (or on next microtask
    // if zone.js is coalescing).
  }
}

```

The actual implementation lives behind the `NgElementStrategy` / `ComponentNgElementStrategy` abstraction (`@angular/elements` source), which is a pluggable strategy interface — Angular ships `ComponentNgElementStrategyFactory` as the default, but the design intentionally allows custom strategies (e.g., a hypothetical AJS/other-framework strategy) to be swapped in.

4.2 Bridging DOM lifecycle to Angular lifecycle

Custom Elements v1 callback	Angular-side effect
<code>constructor()</code>	Nothing Angular-specific yet — just class setup; spec forbids DOM/attribute access here.
<code>connectedCallback()</code>	Component instance created (<code>ngOnChanges</code> with initial inputs → <code>ngOnInit</code> → <code>ngDoCheck</code> → <code>ngAfterContentInit</code> → <code>ngAfterContentChecked</code> → <code>ngAfterViewInit</code> → <code>ngAfterViewChecked</code> , i.e., the full normal first-CD-cycle sequence executes here, driven by Angular's internal <code>createComponent / ApplicationRef.attachView</code>).
<code>attributeChangedCallback()</code>	Sets the corresponding <code>@Input()</code> , which is equivalent to a parent template rebinding an input — this synthesizes an <code>ngOnChanges</code> call (with a <code>SimpleChanges</code> object) followed by a change-detection pass on that component's view.
<code>disconnectedCallback()</code>	Component view detached from <code>ApplicationRef</code> and <code>ngOnDestroy</code> invoked — same teardown as removing the component from an Angular template.
<code>adoptedCallback()</code>	Not specially handled by Angular Elements — extremely rare in practice (only relevant when moving an element to a different <code>Document</code> , e.g., cross-iframe/ <code>importNode</code>).

4.3 Each instance gets its own component instance and (implicit) injector

Every time `connectedCallback()` fires for a *new* tag instance (not a reconnect of the same node), Angular Elements creates a **brand-new Angular component instance**, with its own change-detector, its own `ElementRef`, and dependency resolution rooted at the injector passed into `createCustomElement`. Multiple `<app-popup>` tags on the same page are completely independent component trees sharing only the same root injector (so singleton services registered at that root injector, e.g. `providedIn: 'root'`, are shared across all instances — same singleton semantics as a normal Angular app).

If the component declares its own providers (`providers: [...]` on `@Component`), a child injector is created per-instance just as it would be for a normally-templated component — instance isolation for anything provided at the component level.

4.4 Change detection without a "host" Angular app

Normally, change detection is driven top-down by `ApplicationRef.tick()`, invoked by zone.js patching async APIs (setTimeout, XHR, event listeners) and triggering CD on the whole app after any of them fire. For an Angular Element, `ApplicationRef` still exists (created via `createApplication()` or the `NgModule` bootstrap), and each element's component view is `attachView`-ed to it individually. So:

- With **zone.js** loaded: any async task anywhere that zone.js patches (a click inside the element, a timer, etc.) still triggers `ApplicationRef.tick()`, which runs CD across every attached view, including every Angular Element on the page.
- With **zoneless** change detection (signals-based `ChangeDetectionStrategy.OnPush` + signal inputs), CD is instead driven by explicit signal/marker-based scheduling, which is more efficient for elements embedded in a foreign, non-Angular-CD-aware host page, since you don't need zone.js patching arbitrary global APIs that the host page also relies on.

5. Edge Cases & Gotchas

- **Bundle size — one Angular runtime per element.** Unless you architect shared externals (e.g., loading `@angular/core / zone.js` once globally and marking them as externals in the build for every element bundle), each independently-built Angular Element ships its own copy of the Angular runtime (~50–150 KB gzipped baseline, more with common/forms/router if inadvertently pulled in). Ten different Angular Elements built independently and dropped onto one page can mean ten redundant copies of core Angular code. Mitigations: a shared "vendor" bundle loaded once, module federation, or zoneless + tighter tree-shaking with esbuild's `output: 'static'` application builder.
- **String-only attribute coercion.** HTML attributes are always strings; Angular Elements does **not** parse `"42"` into a number or `"false"` into a boolean automatically via the attribute path. `<my-el count="5">` gives `component.count === "5"` (string) unless the `@Input()` itself declares a transform (`@Input({ transform: numberAttribute })` in newer Angular versions) or the consumer sets the DOM property directly (`e1.count = 5`). This is a very common production bug: `if (this.disabled)` being true because the attribute string was `"false"`.
- **Global CSS bleed without Shadow DOM.** Without `ViewEncapsulation.ShadowDom`, Angular's default emulated encapsulation only prevents the *element's own* styles from leaking out via specificity tricks —

but it does NOT protect against the host page's global styles (e.g., a CSS reset, `* { box-sizing: ... }`, Bootstrap's global button styles) bleeding *into* the Angular Element, since the DOM nodes are still in the same document, not an isolated shadow tree. Only `ViewEncapsulation.ShadowDom` gives real two-way isolation (with the caveat that global custom fonts/CSS variables also won't cross into the shadow tree unless deliberately allowed, e.g., via CSS custom properties, which do pierce Shadow DOM boundaries by design).

- **Event communication back to the host.** `@Output()` becomes a `CustomEvent`; a common mistake is expecting the host to use Angular's `(eventName)` binding syntax — a non-Angular host must use `addEventListener('eventName', handler)`, and payload lives in `event.detail`, not the event object directly. Also: `CustomEvent`s do **not bubble** by default (`bubbles: false`) unless you configure it, which surprises devs trying to catch the event on an ancestor rather than directly on the tag.
- **Multiple Angular versions on one page.** Because each Angular Element is normally self-contained, you genuinely can run an Angular 14-built element and an Angular 17-built element side by side on the same non-Angular host page — each with its own bundled Angular runtime, its own `Zone.js` (careful: only one `Zone.js` global patch can exist safely; two different `zone.js` versions patching the same globals is a classic source of "Zone already loaded" errors or double-invoked callbacks). This is a real and reasonably common use case for gradual internal upgrades of shared widgets across teams that consume them at different cadences — but it demands deliberate build isolation (no shared `zone.js`, or zoneless elements) to avoid collisions.
- **`entryComponents` legacy concern.** Pre-Ivy (Angular <9), any component passed to `createCustomElement` had to be listed in the module's `entryComponents` array because it's never referenced from a template, so the AOT compiler wouldn't otherwise know to generate a component factory for it. Since Ivy (Angular 9+), `entryComponents` is no longer needed/deprecated — components are always locally compilable — but this is a frequent "gotcha" for anyone maintaining older Angular Elements code or reading legacy tutorials.
- **SSR / server contexts.** Custom Elements rely on browser DOM APIs (`customElements`, `HTMLElement`, `shadowRoot`) that don't exist in Node.js by default; Angular Elements is a browser/client-side-only concept — it is not something Angular Universal (SSR) renders on the server without a DOM shim, and typically Angular Elements is registered/hydrated client-side after the page loads.
- **Testing complexity.** Because the element is created outside the normal `TestBed`-driven component tree, testing an Angular Element in isolation typically means using `document.createElement('tag-name')`, appending it to a test DOM, and asserting on its rendered shadow/light DOM and dispatched `CustomEvent`s — a different testing posture than typical `ComponentFixture`-based unit tests.

6. Interview Questions & Answers

Q1. What is Angular Elements, in one sentence?

A: Angular Elements (`@angular/elements`) is the Angular package that lets you package any Angular component as a standards-compliant native Custom Element (Web Component) via the `createCustomElement()` API, so it can be used as a plain HTML tag in any web page, framework, or CMS — not just inside an Angular application.

Q2. What does `createCustomElement()` return, and what do you do with it?

A: It returns an `NgElementConstructor<P>` — a class extending `HTMLElement` that implements the Custom Elements v1 lifecycle callbacks. You register it with the browser's native registry via `customElements.define('tag-name', constructorReturnedByCreateCustomElement)`. After that, the browser itself instantiates the class whenever it parses/creates that tag — you never call `new` on it yourself in normal application code.

Q3. How do HTML attributes map to Angular `@Input()` properties in an Angular Element?

A: Angular Elements introspects the component's `@Input()` metadata and derives a kebab-case attribute name for each (`userId` → `user-id`), exposing them via the generated class's static `observedAttributes`. When the browser calls `attributeChangedCallback` for one of those attribute names, Angular Elements converts the name back to camelCase and assigns the new value (as a raw string) onto the component instance, then triggers change detection — functionally equivalent to a template rebinding an input.

Interviewer intent: This checks whether the candidate understands that this is attribute-string based, not property-based, and can lead into the "string coercion" gotcha — a strong signal of hands-on experience versus textbook familiarity.

Q4. Why would setting `<my-el enabled="false">` in markup NOT disable the component the way you'd expect?

A: Because HTML attributes are always strings — Angular Elements passes the literal string `"false"` to the `@Input()`, and in JavaScript a non-empty string is truthy, so `if (this.enabled)` evaluates to `true`. There is no automatic string→boolean coercion via the attribute path. The fix is either to set the actual boolean via the DOM property (`e1.enabled = false`), or to have the `@Input()` use Angular's input transform feature (e.g., `booleanAttribute`/`numberAttribute` transforms) to coerce the incoming string explicitly.

Q5. How do `@Output()` EventEmitters surface to a non-Angular consumer of the element?

A: They are dispatched as native `CustomEvent` objects on the host DOM element, using the output's property name as the event type. The emitted value is placed in `event.detail`. A plain JS/React/Vue host listens with standard `element.addEventListener('outputName', e => ... e.detail ...)` — Angular's `(eventName)` template binding syntax isn't available because the host page may have no Angular at all.

Q6. Walk me through what happens, internally, from `document.body.appendChild(e1)` to the component being visible on screen.

A: Appending the element to a live document triggers the browser's `connectedCallback()`. Angular Elements' internal `NgElementStrategy` (the default being `ComponentNgElementStrategyFactory`) lazily creates a new Angular component instance the first time this happens, using `createComponent` /the injector supplied to `createCustomElement`. It runs the initial lifecycle sequence (`ngOnChanges` for any inputs already set → `ngOnInit` → `ngDoCheck` → content/view hooks), attaches the component's host view to the shared `ApplicationRef` so it participates in future change-detection ticks, and inserts the component's rendered DOM (into `shadowRoot` if `ViewEncapsulation.ShadowDom` is set, otherwise directly as light-DOM children) under the custom element node.

Interviewer intent: Distinguishes candidates who've only used the API from those who understand the `NgElementStrategy` abstraction and can reason about why lifecycle timing matches "components in a real Angular tree."

Q7. Does each `<my-el>` tag on the page share one Angular component instance, or get its own?

A: Each connected instance of the custom element gets its own, fully independent Angular component instance (its own change detector, `ElementRef`, and, if the component declares its own `providers`, its own child injector). All instances do share the single root injector passed into `createCustomElement`, so root-provided singleton services (`providedIn: 'root'`) are the same instance across every tag on the page — identical semantics to how a normal Angular app shares root-level services across multiple template usages of the same component.

Q8. What's the practical difference between building an Angular Element from an `NgModule` with `DoBootstrap` versus a standalone component with `createApplication()`?

A: Both ultimately need an `Injector` and an `ApplicationRef` context to hand to `createCustomElement`, but they get there differently. The classic `NgModule` approach implements `DoBootstrap.ngDoBootstrap()` on the root module and deliberately skips calling `bootstrapModule`'s normal root-component bootstrap — instead it calls `createCustomElement` inside `ngDoBootstrap` and registers the tag there. The modern standalone approach calls `createApplication()` directly (no `NgModule` needed at all), which returns an `ApplicationRef`/injector without mounting any component, and you call `createCustomElement` on that. The standalone path is simpler, avoids the (legacy, pre-ivy) `entryComponents` requirement entirely, and tree-shakes better since there's no module scaffolding.

Q9. Why might you deliberately use `ViewEncapsulation.ShadowDom` for an Angular Element even though `Emulated` is Angular's default?

A: Because Emulated encapsulation only fakes scoping via generated attribute selectors within the *same* document tree — it stops the element's own styles from leaking out but does nothing to stop the host page's global CSS (resets, third-party frameworks, ID/class collisions) from leaking in. `ShadowDom` gives the element a genuine `shadowRoot`, providing real two-directional CSS and (mostly) DOM isolation — which matters enormously when you don't control, or can't predict, the CSS environment of the host page (e.g., an unknown CMS template or a client's legacy site).

Q10. What issues arise from running two Angular Elements, built from different Angular major versions, on the same page?

A: Since each is typically bundled independently with its own full Angular runtime, they can coexist as long as global, page-singleton resources aren't double-registered — the most common collision is `zone.js`, since it monkey-patches global browser APIs once; loading two different `zone.js` versions/instances on one page throws "Zone already loaded" errors or causes unpredictable double change-detection triggers. Mitigations include ensuring only one bundle loads `zone.js` (share it as an external/global, load it once before either element script), or building the elements zoneless (signal-based change detection) so neither needs the global monkey-patching at all.

Interviewer intent: Tests whether the candidate has actually shipped Elements in a real multi-team/micro-frontend setting versus only having read the docs — this scenario rarely shows up outside of production experience.

Q11. Is Angular Elements suitable for SSR (server-side rendering)? Why or why not?

A: Not directly. Custom Elements rely on browser-only APIs (`customElements`, `HTMLElement`, Shadow DOM) that don't exist in a Node.js SSR environment without a DOM shim/polyfill (e.g., a JSDOM-based approach), and the whole point of Angular Elements — being instantiated by the browser's native parser as it builds the DOM — doesn't have a server-side analog. In practice, Angular Elements is treated as a client-side-only enhancement: the host page (which may itself be server-rendered by any stack) loads the element's script, and the custom element registers and upgrades once the browser's own DOM/JS engine runs, post-load.

Q12. What's the difference between an Angular Element and just exporting an Angular component from a library for another Angular app to import via `NgModule`/standalone `imports`?

A: Importing a component into another Angular app's module/ `imports` array only works if the *consumer* is also an Angular application — it requires the Angular compiler, template syntax bindings, and a shared Angular version/build pipeline. An Angular Element, by contrast, is compiled down to a self-contained package that exposes a plain, framework-agnostic HTML tag with attribute/property/event contracts understood by any DOM, making it consumable from React, Vue, static HTML, or a completely different Angular major version, at the cost of a heavier bundle (its own embedded Angular runtime) and losing template-binding ergonomics (no structural directives across the boundary, string-based attributes, etc.).

Interviewer intent: Probes whether the candidate can articulate the actual tradeoff (interop reach vs. bundle size/ergonomics) rather than reciting "it's a web component" without grounding it in when you'd choose one over the other.

Q13. How would you pass a complex object (not a primitive) into an Angular Element from a non-Angular host?

A: Not via an HTML attribute — attributes are always strings, so you'd have to serialize/deserialize (e.g., `JSON.stringify` into the attribute and `JSON.parse` inside an input setter), which is brittle and lossy for things like functions or class instances. The correct approach is to set the DOM property directly in JavaScript:

```
document.querySelector('my-el').someObjectInput = { foo: 'bar' };
```

Because `attributeChangedCallback` only fires for attribute mutations, not property assignments, Angular Elements' generated class typically also defines a property setter/getter (via `Object.defineProperty` on the input name) that intercepts direct property writes and forwards them into the component instance with full type fidelity, then triggers change detection — this is what lets `e1.someObjectInput = {...}` work correctly.

7. Quick Revision Cheat Sheet

- **API surface:** `createCustomElement(component, { injector })` → `NgElementConstructor`; register with `customElements.define(tagName, ctor)`.
- **Bootstrapping without mounting a root component:** standalone → `createApplication()`; `NgModule`-based → implement `DoBootstrap.ngDoBootstrap()`, skip `bootstrapModule`'s default root mount.
- **Attributes** → `@Input()`: kebab-cased automatically; values always arrive as **strings** via `attributeChangedCallback` — no auto coercion to number/boolean/object. Use the DOM **property** setter for non-string types, or `@Input({ transform: ... })` for coercion.
- `@Output()` → `CustomEvent`: listen via `addEventListener(name, e => e.detail)`; native events don't bubble unless configured.
- **Lifecycle mapping:** `constructor` (bare) → `connectedCallback` (creates component, runs `ngOnInit` + CD, attaches to `ApplicationRef`) → `attributeChangedCallback` (sets input, synthesizes `ngOnChanges` + CD) → `disconnectedCallback` (`ngOnDestroy`, detach from `ApplicationRef`) → `adoptedCallback` (rarely relevant).
- **Instance isolation:** every connected tag = its own component instance + own change detector; shares only the root injector (so root-provided singletons are shared; component-level `providers` are per-instance).
- **Encapsulation:** default `Emulated` doesn't block global host CSS bleeding in; use

`ViewEncapsulation.ShadowDom` for true two-way isolation via a real `shadowRoot`.

- **Bundle cost:** each independently built element typically ships its own full Angular runtime — plan shared externals/module federation/zoneless builds to avoid bloat and `zone.js` double-load collisions.
- **Multi-version coexistence:** possible, but the shared global `zone.js` patch is the main collision risk — mitigate via a single shared `zone.js` or zoneless elements.
- **Pre-ivy legacy note:** `entryComponents` was required for elements not referenced in any template; obsolete since Ivy (Angular 9+).
- **Not SSR-friendly:** Custom Elements APIs are browser-only; Angular Elements upgrades client-side after page load.
- **Primary use cases:** embedding Angular widgets in non-Angular apps, legacy/CMS integration, framework-agnostic design-system distribution, incremental micro-frontend composition.

Created By - Durgesh Singh